

FLAW 1: SQL injection (OWASP 2021 #4)

Link to flaw:

<https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L109>

Description:

SQL injection is a cyber-attack technique where code is injected into an application to obtain data from a database that should not be available for the user. SQL injections specifically target databases operated through the SQL query language. SQL injections are made possible by the application allowing the user to manipulate the SQL queries directly. Typically, dangerous queries can occur when the application allows the user to directly manipulate the query string without any kind of validation. In this case, it is possible to manipulate the query so that the query will ignore key parts of the original query, for example by using SQL comments, or so that the original query conditionals are ignored. SQL injections are dangerous because they can give away valuable information stored in databases such as user information and data, passwords or admin information.

The example Notes application allows using SQL injections to obtain notes stored by other users that should be hidden. The application uses the sqlite3 to connect to the database and makes queries using strings directly manipulatable by the user through the Search functionality in the application. For example, the attacker can input the string “%' --” to the search bar and the application will ignore the owner check in “AND owner_id={user_id}” part of the full query “SELECT id, title FROM notes_note WHERE title LIKE '%{query}%' AND owner_id={user_id}”. This will then result in the Search returning all the notes stored in the database, including ones that should not be shown to the user.

Fix:

The SQL injection flaw can be easily fixed by the ORM functionalities of Django instead of raw SQL string queries. The safe search function uses Note.objects.filter function to query the database. This query cannot be directly manipulated making it safe to use in the application. To fix, comment out line <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L109> and uncomment <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L112>

FLAW 2: CSRF (not in OWASP 2021)

Link to flaw:

<https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L40>

Description:

The cross-site request forgery (CSRF) attack involves using a secondary HTTP request on another site to make malicious requests in the application. For example, an attacker can lure a user to make a POST request in the application that includes malicious information determined by the attacker. CSRF makes it look like the victim purposefully made the request despite not knowing about it. This can happen, for example when the victim has the original application open on their browser and clicks a button on another site that will make the malicious POST request on the application.

Typically, modern web frameworks do not allow making code that allows CSRF to happen. However, the example note application uses the `csrf_exempt` decorator in user register functionality and does not check for a CSRF token that would identify that the user is making the request themselves. This allows performing the CSRF attack. The example code includes the program `csrf_attack.py` that runs a small web applications advertising a monetary prize. If an application user clicks on the page, it will send a POST request that will make a new user on the application that can be used to impersonate the user.

Fix:

The CSRF vulnerability can be fixed by not using the `csrf_exempt` decorator on the registration function and by including a CSRF token check in the HTML code. The token checks that it is the same user performing the request and prevents the POST action if the tokens do not match. To fix, comment out line <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L40> and uncomment https://github.com/KKolehmainen/security-flaws/blob/f06fb8ed6ba77da7bb4f5fab56551b1a9ae3fb85/notes/templates/notes/create_note.html#L6

FLAW 3: Broken Access Control (OWASP 2021 #1)

Link to flaw:

<https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L32>

Description:

Broken access control refers to flaws in the design of the application that allow users to access resources that they should not be able to access. This typically includes omitting code that checks for user rights or checking the incorrect rights. Modern web frameworks provide many tools for easily checking these, but it is still the responsibility of the programmer to implement these.

The example Notes application has a flaw in the `get_object_or_404` function that fetches a specific note. The user is not supposed to be able to access the notes of other users, but the programmer has forgotten to check that the note that the user is accessing actually belongs to the user. Now the function will just fetch the note without checking if it belongs to the user.

Fix:

The flaw is fixed by adding the owner check in the `get` function call:

`get_object_or_404(Note, pk=note_id, owner=request.user)`. Now the function will return a 404 error if the user is not the owner of the note. This fixes the broken access control flaw.

To fix, comment out line <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L32> and uncomment line <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L35>

FLAW 4: Vulnerable and Outdated Components (OWASP 2021 #6)

Link to flaw:

<https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L10>

Description:

Applications typically utilize code that is a part of a separate library or module. Sometimes these components contain other security flaws that are propagated into the application itself just by utilizing these libraries. This becomes increasingly likely if the application uses old libraries that do not contain new fixes for the flaws.

This flaw is simulated in the Notes application by using a database module db_v1 that simulates an old version of the database module. The module contains the function search_notes that is an old implementation of the database search containing the flawed SQL query function that allows SQL injections from FLAW 1.

Fix:

The flaw is fixed by using the newer version of the database module db_v2. Here the search_notes function uses the ORM implementation fixing the SQL injection (FLAW 1 fix). The flaw is corrected simply by importing the new module instead of the old one. To fix, comment out line <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L10> and uncomment line <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L13>

(notice that fixing both flaws 1 and 4 simultaneously will fail, because the fixes depend on each other, fixing flaw 4 will also fix flaw 1)

FLAW 5: Security Misconfiguration (OWASP 2021 #5)

Link to flaw (fix in this case):

<https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L89>

Description:

Security misconfiguration refers to any kind of flaw in the configuration of the application security features. This can include for example using default password, application debug mode in production or providing the secret key in public.

The Notes application does not validate passwords in any way when registering new users. This allows users to, for example, create empty password either purposefully or by accident. This is dangerous because an attacker who knows that there is no validation can easily try to access many accounts without the password. The attacker can create a script that will try to login using an empty password for many usernames quickly.

Fix:

The flaw is fixed by validating the password when registering. Here we validate that the password cannot be empty, but in reality, it is useful to also validate that the password is

strong, for example by requiring certain characters and minimum password length. To fix, uncomment 3 lines starting at <https://github.com/KKolehmainen/security-flaws/blob/609a1e14729e5348f830259e5f3e029487b3f388/notes/views.py#L89>